

Artem Kustikov

Principal Consultant, KI/DevOps/FullStack Softwareentwickler



[LinkedIn](#) | [GitHub](#) | [Credly](#) | [Microsoft Learning](#)
Wien ▫ Österreich
artem.kustikov@gmail.com ▫ +43 664 9310 6218

Principal Consultant und praxisorientierter Softwarearchitekt mit über 20 Jahren Erfahrung in der Konzeption und durchgängigen Umsetzung skalierbarer, cloud-nativer Systeme. Ich verantworte Architektur und Engineering für **KI/LLM-gestützte Anwendungen, verteilte ereignisgesteuerte Plattformen und Full-Stack-Webprodukte** und übersetze Geschäftsziele in widerstandsfähige, gut steuerbare Lösungen. Schwerpunkte: GenAI und agentische Systeme (LLM-Agenten, Evaluation und Observability, MCP), Solution- und Microservices-Architektur, Cloud-Infrastruktur (AWS, Azure, GCP), DevOps und Platform Engineering (Kubernetes, Terraform, CI/CD) sowie Echtzeit-Datenströme (Kafka, Flink). Ich verbinde technische Führung mit umfassender Lieferererfahrung — Mentoring von Teams, Etablierung von Engineering-Best-Practices und die Bereitstellung skalierbarer Produktionssysteme.

Erfahrung

Aug 2022 - Heute

Principal Consultant, KI/DevOps/FullStack Softwareentwickler

[Machine Learning Reply](#) Wien, Österreich

Stack: Node.js, React, Next.js • Java, Quarkus, Flyway • Python, PydanticAI • PostgreSQL, Cosmos DB, Redis • Apache Kafka, Flink, Confluent • Azure OpenAI, AWS Bedrock, Langfuse, MCP • Terraform, Kubernetes, Helm • AWS, Azure

Solution Architect/AI Engineer - *Self-Service-Plattform für agentische KI*

- End-to-End-Verantwortung für eine Self-Service-Plattform, auf der Fachanwender ohne Programmierkenntnisse einen KI-Agenten im Chat beschreiben, ihn als bearbeitbaren visuellen Workflow auf einer Canvas zurückerkennen, veröffentlichen und mit echtem eingehenden Traffic betreiben — ausgelöst über ein gemeinsames Postfach, einbettbare Webformulare, Live-Telefonate und Cron-Zeitpläne.
- Entwurf der Full-Stack-Architektur: ein Next.js-Portal mit der gesamten serverseitigen Plattformlogik, zwei Python Azure Function Apps (Dispatcher und Executor) und vier Storage Queues als einzige Schnittstellen dazwischen, mit einem gemeinsamen Workflow-JSON-Vertrag für ReactFlow-Canvas, Generierungs-Bot, Dispatcher und Executor — eine neue Fähigkeit bedeutet einen neuen Knotentyp und einen Übersetzer, keine neue Pipeline.
- Implementierung des Agent-Executors auf PydanticAI gegen die Azure OpenAI Responses API mit gestreamter Ausgabe, Retry-/Fehlerklassifizierung, geplanten Läufen und einem Redis-gestützten Human-in-the-Loop-Pause/Resume-Primitiv; Agenten stützen ihre Antworten auf Azure AI Foundry Vector Stores, antworten in der Sprache des Kunden, rufen Plattform-Tools wie die Salesforce-Case-Erstellung und die SAP-Übergabe auf und erreichen Kundensysteme über authentifizierte MCP-Server.
- Aufbau des Echtzeit-Sprachkanals: Routing von Telefonnummern zu Workflows, Browser-WebRTC-Sitzungen mit kurzlebigen Ephemeral Tokens, sodass der Account-Key nie den Server verlässt, anruferseitige Transkription, Wissensuche während des Gesprächs und Übergabe des vollständigen Transkripts an einen Agenten nach dem Anruf.
- Umsetzung der MCP-Integrationen mit vollständigem OAuth2-Authorization-Code-Flow, mit AES-256-GCM unter einem Key-Vault-Datenschlüssel verschlüsselten Tokens, automatischem Refresh und personenbezogenen, an einen einzelnen Workflow gebundenen Zugangsdaten — ein gemeinsamer Server bedeutet nie eine gemeinsame Identität.
- Konzeption der Mandantenfähigkeit über die gesamte Datenschicht — domainbasierte Mandantenauflösung, ein Mandanten-Prädikat in jeder Abfrage und ein Test, der den Build fehlschlagen lässt, sobald eine Repository-Funktion die Mandanten-ID auslöst — sodass eine Codebasis sowohl als mandantenfähiges Portal mit kundenspezifischem Branding, Benutzern und Daten als auch als Single-Tenant-Installation in der Azure-Subscription des Kunden ausgeliefert wird.
- Terraform und GitHub-Actions-Pipelines (OIDC Federated Credentials, Bootstrap des State-Backends), die die Plattform aus einer leeren Subscription aufbauen; das Onboarding eines neuen Kunden besteht aus einem GitHub Environment und einer tfvars-Datei. ADRs, Runbooks sowie Unit- und Integrationstests auf TypeScript- und Python-Seite.

AI/ML Engineer - *GenAI-Demonstratoren für Vertriebs- und Angebotsprozesse*

- Aufbau einer LLM-as-a-Judge-Qualitätsschicht für kundenorientierte Chatbots: Jeder Gesprächsschritt wird in Langfuse erfasst und asynchron durch einen AWS Bedrock LLM-Judge in Kombination mit programmierten Metriken bewertet — 9 Live-Qualitätskennzahlen (Qualität, Kosten, Eskalationsrisiko), einsetzbar mit jedem Chatbot.

Solution Architect/FullStack - *Plattform für Einteilung, Abrechnung und Verträge von Konzertveranstaltern*

- Konzeption und Umsetzung einer mandantenfähigen Plattform für mehrere rechtlich getrennte Gesellschaften und Veranstaltungsformate: Ein Login unterstützt mehrere Rollen mit rollenspezifischen Funktionen; jede Abfrage wird über Zugriffsrechte je Mandant eingegrenzt, die an einen unterzeichneten Vertrag gekoppelt werden können.

- Modellierung wiederkehrender Dienstpläne, die Konzerte mit unveränderlichem Besetzungs-Snapshot erzeugen; Doppelbuchungen werden bereits in der Datenbank verhindert. Ergänzt um . ICS-Bestätigungs-E-Mails sowie PDF-Berichte zu Teilnehmern und Gagen.
- Durchgängig TypeScript auf Bun — GraphQL-Yoga-API, Prisma/PostgreSQL, React/Redux/Mantine-SPA, JWT mit Google- und Apple-OAuth — auf AWS ECS Fargate mit Terraform/Terragrunt, GitHub-Actions-CD und SES-Mailversand über die Task-Rolle.
- Durchgängig mit Claude Code entwickelt — von Anforderungen und Datenmodell über API und UI bis zu Infrastruktur und CI/CD — als Fallstudie für AI-gestützte Entwicklung im Produktionsmaßstab.

Senior DevOps/FullStack - Web-Client für KI-Chatbot-System

- Entwicklung einer robusten und widerstandsfähigen CI/CD-Plattform mit integrierten Unit- und e2e-Tests (Playwright) und Feature-Umgebungen.
- Einführung nahtloser Benutzer-Authentifizierung und rollenbasierter Autorisierung gegen AWS Cognito.
- Unterstützung von kollaborativen Chats, Arbeitsbereichen, Dokumenten basierend auf WebSockets und WebRTC.

Senior Java Entwickler/DevOps - Plattform für einheitliches Fahrzeugdaten-Streaming

- Entwurf und Implementierung der Quarkus Management API (Schemata, Datenbestellungen, Verbindungsinformationen, Streaming-Pipelines, Provisionierungsstatus, Plattform-Claims) sowie des Deployment Service, der Kafka-Topics, ACLs und Identity Pools auf Confluent, Azure-Blob-Container für große Nachrichten und Data-Access-Objekte provisioniert.
- Integration der Plattform mit Confluent Kafka und Schema Registry sowie den Legacy-Fahrzeugdatendiensten des Kunden inklusive Schema-Validierung bei der Aufnahme; Angleichung der Ingestion-API an den Legacy-Vertrag und Korrektur der Kafka-Key-Auswahl für Reihenfolgegarantien.
- Aufbau der Flink-Pipeline-Persistenz mit reaktivem Hibernate und Flyway, gebündelten Statusprüfungen und automatischem Neustart fehlgeschlagener Jobs; Refactoring des zentralen Flink-Jobs auf byteidentische Legacy-Ausgabe, Redis-gestützter Regel-Cache mit vollständiger Testabdeckung und Upgrade der Laufzeit auf Flink 2.2.
- Härtung des Zugriffs: mTLS für Datenkonsumenten mit Zertifikatsverwaltung und Kundendokumentation, In-App-JWT-Validierung hinter einem Off/Shadow/Enforce-Schalter zur Entlastung des API-Gateways sowie Migration des gesamten Storage-Zugriffs von SPN-Credentials auf Managed Identity.
- Verantwortung für JWT-Richtlinien und Kapazität von Azure API Management: issuer-gesteuerte und gecachte externe Prüfungen beseitigten ~4,9 Mio. redundante Aufrufe pro 12 h, ein gemeinsamer Redis-Token-Cache (99 % Trefferquote, Terraform für fünf Umgebungen) hob die Kapazitätsgrenze von 31 Einheiten auf, und angepasstes Diagnose-Sampling senkte die Log-Analytics-Kosten um ~43 k EUR/Monat.
- Aufbau des Migrationstools für Flink-Pipelines beim Umzug auf die neue Kubernetes-Plattform (Export, Rewrite, Import, Image-Kopie, Staging-Bereinigung) mit wiederaufnehmbaren Batches und Fehlerberichten; Codierung von Cilium-Netzwerkrichtlinien, Kyverno-konformen Ressourcen, ArgoCD-Anwendungen und Helm-Pins für die Umgebungen dev, int und pre.
- Leitung von Root-Cause-Analysen bei Produktionsvorfällen mit Regressionstests, Unit- und Integrationstests (Testcontainers, WireMock), Einhaltung der Sicherheitsscans (BlackDuck/ORT) für Abhängigkeiten, Härtung der GitHub-Actions-Workflows (Federated Credentials, keine Klartext-Tokens) sowie Betriebswerkzeuge: Consumer-Lag-Checker, Flink-Job-Monitor, Partition-Limiter.

DevOps Engineer/Senior Entwickler - Online Sales Forecasting Tool

- Refactoring bestehender Microservices in Java (Spring Boot), Python (FastAPI) und R (plumber) zur Skalierung in k8s; CI/CD-Framework auf Basis von GitHub Actions. Migration der Legacy-Infrastruktur von AWS (RDS und EC2 mit CloudFormation) auf einen privaten OpenShift-Cluster mit kontinuierlichem Deployment über Helm-Charts sowie Tekton-Trigger und -Pipelines.

Mai 2018 - Feb 2022

Systemarchitekt/Senior FullStack Softwareentwickler

Intetics Minsk, Belarus

Stack: Node.js (express, restify), ReactJS (TypeScript, redux, lerna, grpc-web), Go lang (GRPC, protobuf), Threedium, Rust, Docker, Kubernetes, Google Cloud Platform, MongoDB, PostgreSQL, Bigtable

Arbeit an großem B2B-Projekt in der Modebranche (#1 auf dem US-Markt), Integrationsaufgaben und Entwicklung einer benutzerdefinierten ETL-Engine (Extract, Transform, Load).

- Profiling und Refactoring von Node.js-Microservices; interaktive UI für das ETL-Tool mit [React Flow](#), [dagre](#) und GRPC.
- Design und Implementierung von [Box.com](#)- und [Dropbox](#)-Konnektoren für die ETL-Engine (Go lang); Integration der excelize-Bibliothek in den XSL-Prozessor samt Behebung mehrerer [Probleme](#) im Bibliothekscode.
- Integration von [Threedium](#) 3D-Modellen mit einer benutzerdefinierten React-Komponente.

Okt 2008 - Mai 2018

Systemarchitekt/Senior Softwareentwickler

EffectiveSoft Minsk, Belarus

Teilnahme an 20+ Projekten, darunter NLP- und Textmining-Tool Intellexer.

Stack: Node.js (express), React, Angular, Ext.js, AWS (EC2, ElasticBeanstalk, RDS, CloudFront), MySQL, Redis, MongoDB, .NET (C#/Managed C++), ASP.NET MVC, C++, COM, WinAPI, ActiveMQ, Python.

- **Crowdfunding-Software** — B2B-Plattform für lokale Unternehmen mit enger Integration des [Dwolla](#) Zahlungssystems: Architektur von Web-Client und Admin-App, CI-Setup (Bitbucket Pipelines, AWS CloudFormation) sowie ein Node.js-Background-Worker für Geldtransfers, Zinsberechnung und Finanzprüfung.

- **Medizin: DICOM/ECG-Parsing und -Analyse** — universeller ECG-Datenlader anstelle duplizierter Formatimplementierungen (Physionet, EFS, ISHNE, HL7), Canvas-basierter Web-Client zum Hochladen und Anzeigen von DICOM-Bildern (Zoom, WL-Transformation, interaktive Größenmessung) sowie ein Intranet-Tool zur Klinik-Personalsynchronisation mit ASP.NET MVC/SignalR.

Okt 2006 - Okt 2008

Senior Softwareentwickler

InventionMachine/[IHS Markit](#) Minsk, Belarus

Stack: .NET, C#, C++, ATL/MFC, JavaScript/AJAX, Java, ColdFusion

Jun 2004 - Sep 2006

Softwareentwickler

[SCAND](#) Minsk, Belarus

Stack: ASP 3.0 (VBScript), JavaScript/AJAX, C#, C++/boost/pthread, Java/Spring, MS SQL Server, Oracle

Dez 2002 - Jun 2004

Postgraduierter Student, Lehrer

Belarusian National Technical University Minsk, Belarus

Arbeit an Robotersimulation und analytischem Programmiersystem – komplexes rechnergestütztes Modellierungssystem zur Simulation realer Industrieroboter und ihrer Umgebung, Berechnung der Roboterkinematik, Kollisionsdetektion und analytische Programmierung. Außerdem als Dozent an der BNTU tätig und mehrere IT-bezogene Kurse unterrichtet: Grundlagen der Computernetzwerke, Mathematische Grundlagen der Roboterprogrammierung.

Fähigkeiten

- **JavaScript Full Stack:** (2002-heute) TypeScript, React/redux, Angular, Node.js, Express, Next.js, REST/GraphQL
- **DevOps:** (2015-heute) Docker, Terraform, Kubernetes, AWS, Azure, GCP, Gitlab
- **Python:** (2008-heute) Django, Flask, FastAPI, SQLAlchemy, Celery, NumPy, Pandas, nltk, Seaborn, Pytorch, scikit-learn
- **Java:** (2004-heute) Java 1.3/21, Struts/FOP/JSTL/POI, JBoss, Tomcat, Spring Framework, Spring Boot, Quarkus, Gradle
- Durchführung von 500+ technischen Interviews in JavaScript, DevOps, Java, .NET und Golang

Open-Source-Projekte

- **KateChat** — Selbst-gehostete Multi-Provider-LLM-Chat-Plattform (offene ChatGPT-Alternative). React/TypeScript-Frontend mit Node.js- und Rust-Backend; integriert AWS Bedrock, OpenAI und Yandex AI mit RAG (Docling), MCP-Tool-Servern, In-Browser-Python (Pyodide) und Bildgenerierung. GraphQL-API, WebSocket-Subscriptions, PostgreSQL/Redis.
- **docling.rs** — Performante Rust-Neuimplementierung von Python Docling, aufgenommen in das offizielle Docling-Projekt, die über 20 Dokumentformate (PDF, DOCX, PPTX, XLSX, HTML, EPUB, Bilder, Audio) in eine einheitliche **DoclingDocument**-Struktur für KI/RAG-Pipelines umwandelt. Reiner Rust-PDF-Parser mit ONNX-Layout/TableFormer/OCR-Stack und Whisper-ASR; 2–57× weniger Speicher und bis zu 46× schneller als Python, mit Node.js/TypeScript-Bindings. Erzeugt Markdown, docling-JSON und DocLang (.doclx) und enthält ein modulares RAG-Subsystem: Chunking, austauschbare Embedder und Vektordatenbanken, Hybrid-/Multi-Query-/HyDE-Retrieval sowie eine REST-API.

Ausbildung

Belarusian National Technical University | Minsk, Belarus 1997-2002 | Informatik, Robotik

Zertifikate

- Jun 2026: [Microsoft AI & ML Engineering](#)
- Jan 2026: [AWS Generative AI Applications](#)
- Okt 2025: [Confluent Certified Developer for Apache Kafka](#)
- Mai 2024: HashiCorp Certified: [Terraform Associate \(003\)](#)
- Okt 2023: [AWS Certified Solutions Architect – Associate](#)
- Aug 2022: [DevOps on AWS](#)
- Apr 2017: [Machine Learning and Data Analysis from MIPT/Yandex](#)
- Sep 2016: [Microsoft Certified Solutions Associate: Web Applications](#)
- Feb 2013: Microsoft Certified Solutions Developer: Web Applications

Sprachen

Russisch (Muttersprache) • Englisch (verhandlungssicher), IELTS 6.5, CEFR B2 • Deutsch (beruflich), OIF Integrationsprüfung B1